计算机科学基础 · 模块 04

哈希表与散列原理

从键值映射到冲突解决，理解现代程序中最常用的高效数据结构

课程编号

CS-204 · 数据结构

授课对象

计算机专业二年级

主讲教师

［教师姓名］

课时安排

2 课时（90 分钟）

难度等级

★★★☆☆ 中等

编制日期

2026 春季学期

本讲概要

哈希表是一种以"近乎常数时间"完成查找、插入与删除的关键数据结构，广泛应用于数据库索引、缓存系统、编译器符号表等场景。本讲将从哈希函数的设计原理出发，系统讲解负载因子、冲突解决策略（链地址法与开放寻址法）以及动态扩容机制，并通过电话簿、单词计数等真实案例帮助学生建立从理论到工程实践的完整认知。

01

SECTION ONE · 基础认知

为什么需要哈希表

想象在一本 10 万人的电话簿中查找某个人——若逐页翻找需要遍历全部记录；哈希表让我们"直接翻到那一页"，把查找时间从线性降到接近常数。

本讲学习目标

- ✓ 理解哈希表的核心思想与时间复杂度优势
- ✓ 区分链地址法与开放寻址法两类冲突策略
- ✓ 能在真实编程问题中正确选用哈希结构
- ✓ 掌握哈希函数的设计原则与常见方法
- ✓ 计算负载因子并解释动态扩容的必要性
- ✓ 了解哈希在工程系统中的典型应用场景

1.1 核心思想：键到地址的映射

不同于数组通过**下标**访问、链表通过**遍历**访问，哈希表引入**哈希函数**，把任意键（key）转换为数组中的存储位置（index），实现"通过键直接定位值"。

核心定义

哈希表（Hash Table）

一种基于**键值对（key-value）**存储的数据结构，借助哈希函数将键映射到底层数组的索引位置，平均情况下以 O(1) 完成查找、插入与删除。

$$index = hash(key) \bmod N$$

哈希函数将键映射为 [0, N−1] 区间内的数组下标，N 为底层数组容量

1.2 时间复杂度对比

下表对比哈希表与其他常见结构的平均时间复杂度，可直观看出其在查找密集型场景中的优势：

数据结构	查找	插入	删除	有序性
无序数组	O(n)	O(1)	O(n)	否
有序数组	O(log n)	O(n)	O(n)	是
链表	O(n)	O(1)	O(n)	否
平衡二叉树	O(log n)	O(log n)	O(log n)	是
哈希表	O(1)*	O(1)*	O(1)*	否

! **注意**

表中 O(1)* 表示"平均情况"，哈希冲突严重时会退化到 O(n)，因此哈希函数质量与负载因子控制至关重要。

记忆　**一句话记忆：**哈希表用"空间换时间"——通过哈希函数把查找摊薄成一次计算，实现接近常数的访问。

02 SECTION TWO · 核心机制

哈希函数与冲突解决

2.1 好的哈希函数应具备什么

哈希函数的目标是把键"均匀且确定地"散布到数组中。一个优秀的哈希函数应满足以下性质：

1. **确定性：**相同的键必须始终映射到相同的索引，否则无法再次找到数据。
2. **均匀分布：**不同的键应尽可能均匀地分散，避免大量键挤在同一位置。
3. **高效计算：**函数本身的计算开销应足够低，否则会抵消查找速度的优势。
4. **雪崩效应：**输入的微小变化应导致输出的显著不同，减少规律性冲突。

除留余数法

最常见的方法，取键对容量 N 取模。建议 N 选用**质数**以减少周期性冲突。

```
# 除留余数法
idx = key % N
# N 取质数效果更好
```

乘法散列法

用键乘以常数 A（0<A<1）取小数部分，再乘以 N，对 N 的取值不敏感。

```
# 乘法散列
idx = floor(N * (key*A % 1))
# A≈0.618 黄金比
```

2.2 冲突：当两个键撞在一起

由于键空间通常远大于数组容量，**不同的键映射到相同索引**不可避免，这种现象称为**哈希冲突（collision）**。如何优雅地解决冲突，是哈希表设计的核心。

方案一 · 链地址法（Separate Chaining）

每个数组槽位存储一个链表，冲突的元素依次挂在同一链表上。查找时先定位槽位，再在链表中线性比较。实现简单、扩容友好，是 Java HashMap 等主流实现采用的策略。

✓ **优点**

删除操作简单，负载因子可大于 1；当链表过长时（如超过 8 个节点）可转换为红黑树，把最坏查找从 O(n) 优化到 O(log n)。

方案二 · 开放寻址法（Open Addressing）

所有元素都存放在数组内部，发生冲突时按某种探测序列寻找下一个空槽。常见探测方式有线性探测、二次探测与双重哈希。

探测方式	探测公式（第 i 次）	主要问题
线性探测	$(h + i) \bmod N$	易产生连续聚集
二次探测	$(h + i^2) \bmod N$	缓解一次聚集
双重哈希	$(h_1 + i \cdot h_2) \bmod N$	分布最均匀

⚠ **常见误区**

开放寻址法中**不能简单地把删除的槽位置空**，否则会中断探测链。正确做法是使用"墓碑标记（tombstone）"标识已删除位置。

SECTION THREE · 实践应用

03 负载因子、扩容与案例

3.1 负载因子与动态扩容

负载因子（Load Factor）衡量哈希表的"拥挤程度"，是决定何时扩容的关键指标：

$$\alpha = n / N$$

n 为已存元素个数，N 为数组容量；α 越大冲突概率越高

当 α 超过预设阈值（链地址法常取 0.75，开放寻址法常取 0.5），就触发**扩容（rehashing）**：分配更大数组（通常翻倍）并重新哈希所有元素。扩容单次代价 O(n)，但分摊到每次插入仍是 O(1)。

重点

权衡关系：负载因子越小，冲突越少、查找越快，但空间浪费越多；越大则空间利用率越高，但冲突风险上升。0.75 是时间与空间的经典折中。

3.2 实战案例

案例 A 用哈希表统计单词频率

问题：给定一篇文章，统计每个单词出现的次数。

思路：以单词为键、出现次数为值。遍历文本，键已存在则计数 +1，否则插入置 1。

```
for word in text.split():
  counts[word] = counts.get(word, 0) + 1
```

复杂度：遍历 O(m)，每次哈希操作平均 O(1)，总体 **O(m)**，m 为单词总数。

案例 B 两数之和（LeetCode 经典题）

问题：在数组中找出两个数，使其和等于目标值 target。

思路：遍历数组，对每个 x 在哈希表中查找 target−x，把"嵌套循环 $O(n^2)$"优化为"一次遍历 O(n)"。

```
for i, x in enumerate(nums):
  if target-x in seen: return [seen[target-x], i]
  seen[x] = i
```

3.3 工程中的应用场景

- **数据库索引：**等值查询的快速定位
- **缓存系统：**Redis、Memcached 键值存储
- **编译器：**符号表管理变量与函数名
- **去重与集合：**HashSet 的 O(1) 判断

Σ 本讲小结

01 哈希函数实现键到地址的直接映射，平均 O(1) 增删查

02 优秀哈希函数需确定、均匀、高效并具雪崩效应

03 链地址法与开放寻址法是两类主流冲突解决方案

04 负载因子控制拥挤度，超阈值触发扩容维持性能

04 SECTION FOUR · 巩固训练

课后练习与思考

一、选择题 每题 5 分

1 哈希表在平均情况下，查找操作的时间复杂度为？

A. O(n)　　B. O(log n)

C. O(1)　　D. $O(n^2)$

2 下列关于负载因子 α 的说法，正确的是？

A. α 越大查找越快　　B. $\alpha = n / N$

C. α 与冲突无关　　D. α 恒等于 1

3 开放寻址法删除元素时，正确的处理方式是？

A. 直接置空　　B. 使用墓碑标记

C. 移动后续元素　　D. 无需处理

二、简答题 每题 10 分

4 请说明链地址法与开放寻址法各自的优缺点，并举例说明适用场景。

5 为什么扩容操作单次代价为 O(n)，但分摊到每次插入仍可视为 O(1)？

三、编程实践 附加题

6 用你熟悉的语言实现一个支持自动扩容的哈希表（链地址法），并编写测试验证其增删查功能。要求负载因子超过 0.75 时自动翻倍扩容。

学习自测清单（完成后请打勾）

- [ ] 我能说出哈希表平均 O(1) 的原因
- [ ] 我能区分链地址法与开放寻址法
- [ ] 我会计算负载因子并判断扩容时机
- [ ] 我能写出至少两种哈希函数方法
- [ ] 我理解墓碑标记的作用
- [ ] 我能举出三个哈希的工程应用

下讲预告

模块 05《图的表示与遍历》——邻接矩阵、邻接表，以及 DFS 与 BFS 两大遍历算法。

延伸阅读

《算法导论》第 11 章 · 散列表；《数据结构与算法分析》第 5 章。

模板说明： 本页所有版块（选择题 / 简答题 / 编程实践 / 自测清单 / 预告）均可按需替换文字、增删题目。复制任意版块代码即可在其他课程讲义中复用。